A3C

arXiv:1602.01783v2 [cs.LG] 16 Jun 2016

# Asynchronous Methods for Deep Reinforcement Learning

**Volodymyr Mnih**[1] VMNIH@GOOGLE.COM
**Adrià Puigdomènech Badia**[1] ADRIAP@GOOGLE.COM
**Mehdi Mirza**[1,2] MIRZAMOM@IRO.UMONTREAL.CA
**Alex Graves**[1] GRAVESA@GOOGLE.COM
**Tim Harley**[1] THARLEY@GOOGLE.COM
**Timothy P. Lillicrap**[1] COUNTZERO@GOOGLE.COM
**David Silver**[1] DAVIDSILVER@GOOGLE.COM
**Koray Kavukcuoglu** [1] KORAYK@GOOGLE.COM

[1] Google DeepMind
[2] Montreal Institute for Learning Algorithms (MILA), University of Montreal

## Abstract

We propose a conceptually simple and lightweight framework for deep reinforcement learning that uses asynchronous gradient descent for optimization of deep neural network controllers. We present asynchronous variants of four standard reinforcement learning algorithms and show that parallel actor-learners have a stabilizing effect on training allowing all four methods to successfully train neural network controllers. The best performing method, an asynchronous variant of actor-critic, surpasses the current state-of-the-art on the Atari domain while training for half the time on a single multi-core CPU instead of a GPU. Furthermore, we show that asynchronous actor-critic succeeds on a wide variety of continuous motor control problems as well as on a new task of navigating random 3D mazes using a visual input.

## 1. Introduction

Deep neural networks provide rich representations that can enable reinforcement learning (RL) algorithms to perform effectively. However, it was previously thought that the combination of simple online RL algorithms with deep neural networks was fundamentally unstable. Instead, a variety of solutions have been proposed to stabilize the algorithm (Riedmiller, 2005; Mnih et al., 2013; 2015; Van Hasselt et al., 2015; Schulman et al., 2015a). These approaches share a common idea: the sequence of observed data encountered by an online RL agent is non-stationary, and online RL updates are strongly correlated. By storing the agent's data in an experience replay memory, the data can be batched (Riedmiller, 2005; Schulman et al., 2015a) or randomly sampled (Mnih et al., 2013; 2015; Van Hasselt et al., 2015) from different time-steps. Aggregating over memory in this way reduces non-stationarity and decorrelates updates, but at the same time limits the methods to off-policy reinforcement learning algorithms.

Deep RL algorithms based on experience replay have achieved unprecedented success in challenging domains such as Atari 2600. However, experience replay has several drawbacks: it uses more memory and computation per real interaction; and it requires off-policy learning algorithms that can update from data generated by an older policy.

In this paper we provide a very different paradigm for deep reinforcement learning. Instead of experience replay, we asynchronously execute multiple agents in parallel, on multiple instances of the environment. This parallelism also decorrelates the agents' data into a more stationary process, since at any given time-step the parallel agents will be experiencing a variety of different states. This simple idea enables a much larger spectrum of fundamental on-policy RL algorithms, such as Sarsa, n-step methods, and actor-critic methods, as well as off-policy RL algorithms such as Q-learning, to be applied robustly and effectively using deep neural networks.

Our parallel reinforcement learning paradigm also offers practical benefits. Whereas previous approaches to deep reinforcement learning rely heavily on specialized hardware such as GPUs (Mnih et al., 2015; Van Hasselt et al., 2015; Schaul et al., 2015) or massively distributed architectures (Nair et al., 2015), our experiments run on a single machine with a standard multi-core CPU. When applied to a variety of Atari 2600 domains, on many games asynchronous reinforcement learning achieves better results, in far less

*Proceedings of the 33rd International Conference on Machine Learning*, New York, NY, USA, 2016. JMLR: W&CP volume 48. Copyright 2016 by the author(s).

time than previous GPU-based algorithms, using far less resource than massively distributed approaches. The best of the proposed methods, asynchronous advantage actor-critic (A3C), also mastered a variety of continuous motor control tasks as well as learned general strategies for exploring 3D mazes purely from visual inputs. We believe that the success of A3C on both 2D and 3D games, discrete and continuous action spaces, as well as its ability to train feedforward and recurrent agents makes it the most general and successful reinforcement learning agent to date.

## 2. Related Work

The General Reinforcement Learning Architecture (Gorila) of (Nair et al., 2015) performs asynchronous training of reinforcement learning agents in a distributed setting. In Gorila, each process contains an actor that acts in its own copy of the environment, a separate replay memory, and a learner that samples data from the replay memory and computes gradients of the DQN loss (Mnih et al., 2015) with respect to the policy parameters. The gradients are asynchronously sent to a central parameter server which updates a central copy of the model. The updated policy parameters are sent to the actor-learners at fixed intervals. By using 100 separate actor-learner processes and 30 parameter server instances, a total of 130 machines, Gorila was able to significantly outperform DQN over 49 Atari games. On many games Gorila reached the score achieved by DQN over 20 times faster than DQN. We also note that a similar way of parallelizing DQN was proposed by (Chavez et al., 2015).

In earlier work, (Li & Schuurmans, 2011) applied the Map Reduce framework to parallelizing batch reinforcement learning methods with linear function approximation. Parallelism was used to speed up large matrix operations but not to parallelize the collection of experience or stabilize learning. (Grounds & Kudenko, 2008) proposed a parallel version of the Sarsa algorithm that uses multiple separate actor-learners to accelerate training. Each actor-learner learns separately and periodically sends updates to weights that have changed significantly to the other learners using peer-to-peer communication.

(Tsitsiklis, 1994) studied convergence properties of Q-learning in the asynchronous optimization setting. These results show that Q-learning is still guaranteed to converge when some of the information is outdated as long as outdated information is always eventually discarded and several other technical assumptions are satisfied. Even earlier, (Bertsekas, 1982) studied the related problem of distributed dynamic programming.

Another related area of work is in evolutionary methods, which are often straightforward to parallelize by distributing fitness evaluations over multiple machines or threads (Tomassini, 1999). Such parallel evolutionary approaches have recently been applied to some visual reinforcement learning tasks. In one example, (Koutník et al., 2014) evolved convolutional neural network controllers for the TORCS driving simulator by performing fitness evaluations on 8 CPU cores in parallel.

## 3. Reinforcement Learning Background

We consider the standard reinforcement learning setting where an agent interacts with an environment $\mathcal{E}$ over a number of discrete time steps. At each time step $t$, the agent receives a state $s_t$ and selects an action $a_t$ from some set of possible actions $\mathcal{A}$ according to its policy $\pi$, where $\pi$ is a mapping from states $s_t$ to actions $a_t$. In return, the agent receives the next state $s_{t+1}$ and receives a scalar reward $r_t$. The process continues until the agent reaches a terminal state after which the process restarts. The return $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ is the total accumulated return from time step $t$ with discount factor $\gamma \in (0, 1]$. The goal of the agent is to maximize the expected return from each state $s_t$.

The action value $Q^\pi(s, a) = \mathbb{E}[R_t | s_t = s, a]$ is the expected return for selecting action $a$ in state $s$ and following policy $\pi$. The optimal value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ gives the maximum action value for state $s$ and action $a$ achievable by any policy. Similarly, the value of state $s$ under policy $\pi$ is defined as $V^\pi(s) = \mathbb{E}[R_t | s_t = s]$ and is simply the expected return for following policy $\pi$ from state $s$.

In value-based model-free reinforcement learning methods, the action value function is represented using a function approximator, such as a neural network. Let $Q(s, a; \theta)$ be an approximate action-value function with parameters $\theta$. The updates to $\theta$ can be derived from a variety of reinforcement learning algorithms. One example of such an algorithm is Q-learning, which aims to directly approximate the optimal action value function: $Q^*(s, a) \approx Q(s, a; \theta)$. In one-step Q-learning, the parameters $\theta$ of the action value function $Q(s, a; \theta)$ are learned by iteratively minimizing a sequence of loss functions, where the $i$th loss function defined as

$$L_i(\theta_i) = \mathbb{E}\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right)^2$$

where $s'$ is the state encountered after state $s$.

We refer to the above method as one-step Q-learning because it updates the action value $Q(s, a)$ toward the one-step return $r + \gamma \max_{a'} Q(s', a'; \theta)$. One drawback of using one-step methods is that obtaining a reward $r$ only directly affects the value of the state action pair $s, a$ that led to the reward. The values of other state action pairs are affected only indirectly through the updated value $Q(s, a)$. This can make the learning process slow since many updates are required the propagate a reward to the relevant preceding states and actions.

One way of propagating rewards faster is by using $n$-step returns (Watkins, 1989; Peng & Williams, 1996). In $n$-step Q-learning, $Q(s,a)$ is updated toward the $n$-step return defined as $r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a)$. This results in a single reward $r$ directly affecting the values of $n$ preceding state action pairs. This makes the process of propagating rewards to relevant state-action pairs potentially much more efficient.

In contrast to value-based methods, policy-based model-free methods directly parameterize the policy $\pi(a|s;\theta)$ and update the parameters $\theta$ by performing, typically approximate, gradient ascent on $\mathbb{E}[R_t]$. One example of such a method is the REINFORCE family of algorithms due to Williams (1992). Standard REINFORCE updates the policy parameters $\theta$ in the direction $\nabla_\theta \log \pi(a_t|s_t;\theta) R_t$, which is an unbiased estimate of $\nabla_\theta \mathbb{E}[R_t]$. It is possible to reduce the variance of this estimate while keeping it unbiased by subtracting a learned function of the state $b_t(s_t)$, known as a baseline (Williams, 1992), from the return. The resulting gradient is $\nabla_\theta \log \pi(a_t|s_t;\theta)\,(R_t - b_t(s_t))$.

A learned estimate of the value function is commonly used as the baseline $b_t(s_t) \approx V^\pi(s_t)$ leading to a much lower variance estimate of the policy gradient. When an approximate value function is used as the baseline, the quantity $R_t - b_t$ used to scale the policy gradient can be seen as an estimate of the *advantage* of action $a_t$ in state $s_t$, or $A(a_t, s_t) = Q(a_t, s_t) - V(s_t)$, because $R_t$ is an estimate of $Q^\pi(a_t, s_t)$ and $b_t$ is an estimate of $V^\pi(s_t)$. This approach can be viewed as an actor-critic architecture where the policy $\pi$ is the actor and the baseline $b_t$ is the critic (Sutton & Barto, 1998; Degris et al., 2012).

## 4. Asynchronous RL Framework

We now present multi-threaded asynchronous variants of one-step Sarsa, one-step Q-learning, n-step Q-learning, and advantage actor-critic. The aim in designing these methods was to find RL algorithms that can train deep neural network policies reliably and without large resource requirements. While the underlying RL methods are quite different, with actor-critic being an on-policy policy search method and Q-learning being an off-policy value-based method, we use two main ideas to make all four algorithms practical given our design goal.

First, we use asynchronous actor-learners, similarly to the Gorila framework (Nair et al., 2015), but instead of using separate machines and a parameter server, we use multiple CPU threads on a single machine. Keeping the learners on a single machine removes the communication costs of sending gradients and parameters and enables us to use Hogwild! (Recht et al., 2011) style updates for training.

Second, we make the observation that multiple actor-learners running in parallel are likely to be exploring different parts of the environment. Moreover, one can explicitly use different exploration policies in each actor-learner to maximize this diversity. By running different exploration policies in different threads, the overall changes being made to the parameters by multiple actor-learners applying online updates in parallel are likely to be less correlated in time than a single agent applying online updates. Hence, we do not use a replay memory and rely on parallel actors employing different exploration policies to perform the stabilizing role undertaken by experience replay in the DQN training algorithm.

**Algorithm 1** Asynchronous one-step Q-learning - pseudocode for each actor-learner thread.

*// Assume global shared $\theta$, $\theta^-$, and counter $T = 0$.*
Initialize thread step counter $t \leftarrow 0$
Initialize target network weights $\theta^- \leftarrow \theta$
Initialize network gradients $d\theta \leftarrow 0$
Get initial state $s$
**repeat**
  Take action $a$ with $\epsilon$-greedy policy based on $Q(s, a; \theta)$
  Receive new state $s'$ and reward $r$
  $y = \begin{cases} r & \text{for terminal } s' \\ r + \gamma \max_{a'} Q(s', a'; \theta^-) & \text{for non-terminal } s' \end{cases}$
  Accumulate gradients wrt $\theta$: $d\theta \leftarrow d\theta + \frac{\partial (y - Q(s,a;\theta))^2}{\partial \theta}$
  $s = s'$
  $T \leftarrow T + 1$ and $t \leftarrow t + 1$
  **if** $T \mod I_{target} == 0$ **then**
    Update the target network $\theta^- \leftarrow \theta$
  **end if**
  **if** $t \mod I_{AsyncUpdate} == 0$ or $s$ is terminal **then**
    Perform asynchronous update of $\theta$ using $d\theta$.
    Clear gradients $d\theta \leftarrow 0$.
  **end if**
**until** $T > T_{max}$

In addition to stabilizing learning, using multiple parallel actor-learners has multiple practical benefits. First, we obtain a reduction in training time that is roughly linear in the number of parallel actor-learners. Second, since we no longer rely on experience replay for stabilizing learning we are able to use on-policy reinforcement learning methods such as Sarsa and actor-critic to train neural networks in a stable way. We now describe our variants of one-step Q-learning, one-step Sarsa, n-step Q-learning and advantage actor-critic.

**Asynchronous one-step Q-learning:** Pseudocode for our variant of Q-learning, which we call Asynchronous one-step Q-learning, is shown in Algorithm 1. Each thread interacts with its own copy of the environment and at each step computes a gradient of the Q-learning loss. We use a shared and slowly changing target network in computing the Q-learning loss, as was proposed in the DQN training method. We also accumulate gradients over multiple timesteps before they are applied, which is similar to us-

ing minibatches. This reduces the chances of multiple actor learners overwriting each other's updates. Accumulating updates over several steps also provides some ability to trade off computational efficiency for data efficiency.

Finally, we found that giving each thread a different exploration policy helps improve robustness. Adding diversity to exploration in this manner also generally improves performance through better exploration. While there are many possible ways of making the exploration policies differ we experiment with using $\epsilon$-greedy exploration with $\epsilon$ periodically sampled from some distribution by each thread.

**Asynchronous one-step Sarsa:** The asynchronous one-step Sarsa algorithm is the same as asynchronous one-step Q-learning as given in Algorithm 1 except that it uses a different target value for $Q(s, a)$. The target value used by one-step Sarsa is $r + \gamma Q(s', a'; \theta^-)$ where $a'$ is the action taken in state $s'$ (Rummery & Niranjan, 1994; Sutton & Barto, 1998). We again use a target network and updates accumulated over multiple timesteps to stabilize learning.

**Asynchronous n-step Q-learning:** Pseudocode for our variant of multi-step Q-learning is shown in Supplementary Algorithm S2. The algorithm is somewhat unusual because it operates in the forward view by explicitly computing n-step returns, as opposed to the more common backward view used by techniques like eligibility traces (Sutton & Barto, 1998). We found that using the forward view is easier when training neural networks with momentum-based methods and backpropagation through time. In order to compute a single update, the algorithm first selects actions using its exploration policy for up to $t_{max}$ steps or until a terminal state is reached. This process results in the agent receiving up to $t_{max}$ rewards from the environment since its last update. The algorithm then computes gradients for n-step Q-learning updates for each of the state-action pairs encountered since the last update. Each n-step update uses the longest possible n-step return resulting in a one-step update for the last state, a two-step update for the second last state, and so on for a total of up to $t_{max}$ updates. The accumulated updates are applied in a single gradient step.

**Asynchronous advantage actor-critic:** The algorithm, which we call asynchronous advantage actor-critic (A3C), maintains a policy $\pi(a_t|s_t; \theta)$ and an estimate of the value function $V(s_t; \theta_v)$. Like our variant of n-step Q-learning, our variant of actor-critic also operates in the forward view and uses the same mix of n-step returns to update both the policy and the value-function. The policy and the value function are updated after every $t_{max}$ actions or when a terminal state is reached. The update performed by the algorithm can be seen as $\nabla_{\theta'} \log \pi(a_t|s_t; \theta') A(s_t, a_t; \theta, \theta_v)$ where $A(s_t, a_t; \theta, \theta_v)$ is an estimate of the advantage function given by $\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v)$, where $k$ can vary from state to state and is upper-bounded by $t_{max}$. The pseudocode for the algorithm is presented in Supplementary Algorithm S3.

As with the value-based methods we rely on parallel actor-learners and accumulated updates for improving training stability. Note that while the parameters $\theta$ of the policy and $\theta_v$ of the value function are shown as being separate for generality, we always share some of the parameters in practice. We typically use a convolutional neural network that has one softmax output for the policy $\pi(a_t|s_t; \theta)$ and one linear output for the value function $V(s_t; \theta_v)$, with all non-output layers shared.

We also found that adding the entropy of the policy $\pi$ to the objective function improved exploration by discouraging premature convergence to suboptimal deterministic policies. This technique was originally proposed by (Williams & Peng, 1991), who found that it was particularly helpful on tasks requiring hierarchical behavior. The gradient of the full objective function including the entropy regularization term with respect to the policy parameters takes the form $\nabla_{\theta'} \log \pi(a_t|s_t; \theta')(R_t - V(s_t; \theta_v)) + \beta \nabla_{\theta'} H(\pi(s_t; \theta'))$, where $H$ is the entropy. The hyperparameter $\beta$ controls the strength of the entropy regularization term.

**Optimization:** We investigated three different optimization algorithms in our asynchronous framework – SGD with momentum, RMSProp (Tieleman & Hinton, 2012) without shared statistics, and RMSProp with shared statistics. We used the standard non-centered RMSProp update given by

$$g = \alpha g + (1 - \alpha)\Delta\theta^2 \text{ and } \theta \leftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g + \epsilon}}, \qquad (1)$$

where all operations are performed elementwise. A comparison on a subset of Atari 2600 games showed that a variant of RMSProp where statistics $g$ are shared across threads is considerably more robust than the other two methods. Full details of the methods and comparisons are included in Supplementary Section 7.

## 5. Experiments

We use four different platforms for assessing the properties of the proposed framework. We perform most of our experiments using the Arcade Learning Environment (Bellemare et al., 2012), which provides a simulator for Atari 2600 games. This is one of the most commonly used benchmark environments for RL algorithms. We use the Atari domain to compare against state of the art results (Van Hasselt et al., 2015; Wang et al., 2015; Schaul et al., 2015; Nair et al., 2015; Mnih et al., 2015), as well as to carry out a detailed stability and scalability analysis of the proposed methods. We performed further comparisons using the TORCS 3D car racing simulator (Wymann et al., 2013). We also use

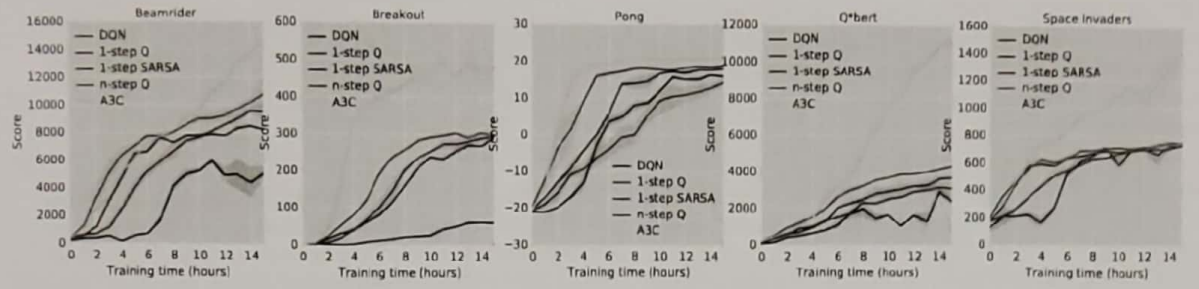



*Figure 1.* Learning speed comparison for DQN and the new asynchronous algorithms on five Atari 2600 games. DQN was trained on a single Nvidia K40 GPU while the asynchronous methods were trained using 16 CPU cores. The plots are averaged over 5 runs. In the case of DQN the runs were for different seeds with fixed hyperparameters. For asynchronous methods we average over the best 5 models from 50 experiments with learning rates sampled from $LogUniform(10^{-4}, 10^{-2})$ and all other hyperparameters fixed.

two additional domains to evaluate only the A3C algorithm – Mujoco and Labyrinth. MuJoCo (Todorov, 2015) is a physics simulator for evaluating agents on continuous motor control tasks with contact dynamics. Labyrinth is a new 3D environment where the agent must learn to find rewards in randomly generated mazes from a visual input. The precise details of our experimental setup can be found in Supplementary Section 8.

### 5.1. Atari 2600 Games

We first present results on a subset of Atari 2600 games to demonstrate the training speed of the new methods. Figure 1 compares the learning speed of the DQN algorithm trained on an Nvidia K40 GPU with the asynchronous methods trained using 16 CPU cores on five Atari 2600 games. The results show that all four asynchronous methods we presented can successfully train neural network controllers on the Atari domain. The asynchronous methods tend to learn faster than DQN, with significantly faster learning on some games, while training on only 16 CPU cores. Additionally, the results suggest that n-step methods learn faster than one-step methods on some games. Overall, the policy-based advantage actor-critic method significantly outperforms all three value-based methods.

We then evaluated asynchronous advantage actor-critic on 57 Atari games. In order to compare with the state of the art in Atari game playing, we largely followed the training and evaluation protocol of (Van Hasselt et al., 2015). Specifically, we tuned hyperparameters (learning rate and amount of gradient norm clipping) using a search on six Atari games (Beamrider, Breakout, Pong, Q*bert, Seaquest and Space Invaders) and then fixed all hyperparameters for all 57 games. We trained both a feedforward agent with the same architecture as (Mnih et al., 2015; Nair et al., 2015; Van Hasselt et al., 2015) as well as a recurrent agent with an additional 256 LSTM cells after the final hidden layer. We additionally used the final network weights for evaluation to make the results more comparable to the original results

| Method | Training Time | Mean | Median |
|---|---|---|---|
| DQN | 8 days on GPU | 121.9% | 47.5% |
| Gorila | 4 days, 100 machines | 215.2% | 71.3% |
| D-DQN | 8 days on GPU | 332.9% | 110.9% |
| Dueling D-DQN | 8 days on GPU | 343.8% | 117.1% |
| Prioritized DQN | 8 days on GPU | 463.6% | 127.6% |
| A3C, FF | 1 day on CPU | 344.1% | 68.2% |
| A3C, FF | 4 days on CPU | 496.8% | 116.6% |
| A3C, LSTM | 4 days on CPU | 623.0% | 112.6% |

*Table 1.* Mean and median human-normalized scores on 57 Atari games using the human starts evaluation metric. Supplementary Table SS3 shows the raw scores for all games.

from (Bellemare et al., 2012). We trained our agents for four days using 16 CPU cores, while the other agents were trained for 8 to 10 days on Nvidia K40 GPUs. Table 1 shows the average and median human-normalized scores obtained by our agents trained by asynchronous advantage actor-critic (A3C) as well as the current state-of-the art. Supplementary Table S3 shows the scores on all games. A3C significantly improves on state-of-the-art the average score over 57 games in half the training time of the other methods while using only 16 CPU cores and no GPU. Furthermore, after just one day of training, A3C matches the average human normalized score of Dueling Double DQN and almost reaches the median human normalized score of Gorila. We note that many of the improvements that are presented in Double DQN (Van Hasselt et al., 2015) and Dueling Double DQN (Wang et al., 2015) can be incorporated to 1-step Q and n-step Q methods presented in this work with similar potential improvements.

### 5.2. TORCS Car Racing Simulator

We also compared the four asynchronous methods on the TORCS 3D car racing game (Wymann et al., 2013). TORCS not only has more realistic graphics than Atari 2600 games, but also requires the agent to learn the dynamics of the car it is controlling. At each step, an agent received only a visual input in the form of an RGB image

of the current frame as well as a reward proportional to the agent's velocity along the center of the track at the agent's current position. We used the same neural network architecture as the one used in the Atari experiments specified in Supplementary Section 8. We performed experiments using four different settings – the agent controlling a slow car with and without opponent bots, and the agent controlling a fast car with and without opponent bots. Full results can be found in Supplementary Figure S6. A3C was the best performing agent, reaching between roughly 75% and 90% of the score obtained by a human tester on all four game configurations in about 12 hours of training. A video showing the learned driving behavior of the A3C agent can be found at `https://youtu.be/0xo1Ldx3L5Q`.

### 5.3. Continuous Action Control Using the MuJoCo Physics Simulator

We also examined a set of tasks where the action space is continuous. In particular, we looked at a set of rigid body physics domains with contact dynamics where the tasks include many examples of manipulation and locomotion. These tasks were simulated using the Mujoco physics engine. We evaluated only the asynchronous advantage actor-critic algorithm since, unlike the value-based methods, it is easily extended to continuous actions. In all problems, using either the physical state or pixels as input, Asynchronous Advantage-Critic found good solutions in less than 24 hours of training and typically in under a few hours. Some successful policies learned by our agent can be seen in the following video `https://youtu.be/Ajjc08-iPx8`. Further details about this experiment can be found in Supplementary Section 9.

### 5.4. Labyrinth

We performed an additional set of experiments with A3C on a new 3D environment called Labyrinth. The specific task we considered involved the agent learning to find rewards in randomly generated mazes. At the beginning of each episode the agent was placed in a new randomly generated maze consisting of rooms and corridors. Each maze contained two types of objects that the agent was rewarded for finding – apples and portals. Picking up an apple led to a reward of 1. Entering a portal led to a reward of 10 after which the agent was respawned in a new random location in the maze and all previously collected apples were regenerated. An episode terminated after 60 seconds after which a new episode would begin. The aim of the agent is to collect as many points as possible in the time limit and the optimal strategy involves first finding the portal and then repeatedly going back to it after each respawn. This task is much more challenging than the TORCS driving domain because the agent is faced with a new maze in each episode and must learn a general strategy for exploring random mazes.

| | Number of threads | | | | |
|---|---|---|---|---|---|
| Method | 1 | 2 | 4 | 8 | 16 |
| 1-step Q | 1.0 | **3.0** | **6.3** | **13.3** | **24.1** |
| 1-step SARSA | 1.0 | **2.8** | **5.9** | **13.1** | **22.1** |
| n-step Q | 1.0 | **2.7** | **5.9** | **10.7** | **17.2** |
| A3C | 1.0 | 2.1 | 3.7 | 6.9 | 12.5 |

*Table 2.* The average training speedup for each method and number of threads averaged over seven Atari games. To compute the training speed-up on a single game we measured the time to required reach a fixed reference score using each method and number of threads. The speedup from using $n$ threads on a game was defined as the time required to reach a fixed reference score using one thread divided the time required to reach the reference score using $n$ threads. The table shows the speedups averaged over seven Atari games (Beamrider, Breakout, Enduro, Pong, Q*bert, Seaquest, and Space Invaders).

We trained an A3C LSTM agent on this task using only $84 \times 84$ RGB images as input. The final average score of around 50 indicates that the agent learned a reasonable strategy for exploring random 3D maxes using only a visual input. A video showing one of the agents exploring previously unseen mazes is included at `https://youtu.be/nMR5mjCFZCw`.

### 5.5. Scalability and Data Efficiency

We analyzed the effectiveness of our proposed framework by looking at how the training time and data efficiency changes with the number of parallel actor-learners. When using multiple workers in parallel and updating a shared model, one would expect that in an ideal case, for a given task and algorithm, the number of training steps to achieve a certain score would remain the same with varying numbers of workers. Therefore, the advantage would be solely due to the ability of the system to consume more data in the same amount of wall clock time and possibly improved exploration. Table 2 shows the training speed-up achieved by using increasing numbers of parallel actor-learners averaged over seven Atari games. These results show that all four methods achieve substantial speedups from using multiple worker threads, with 16 threads leading to at least an order of magnitude speedup. This confirms that our proposed framework scales well with the number of parallel workers, making efficient use of resources.

Somewhat surprisingly, asynchronous one-step Q-learning and Sarsa algorithms exhibit superlinear speedups that cannot be explained by purely computational gains. We observe that one-step methods (one-step Q and one-step Sarsa) often require less data to achieve a particular score when using more parallel actor-learners. We believe this is due to positive effect of multiple threads to reduce the bias in one-step methods. These effects are shown more clearly in Figure 3, which shows plots of the average score against the total number of training frames for different

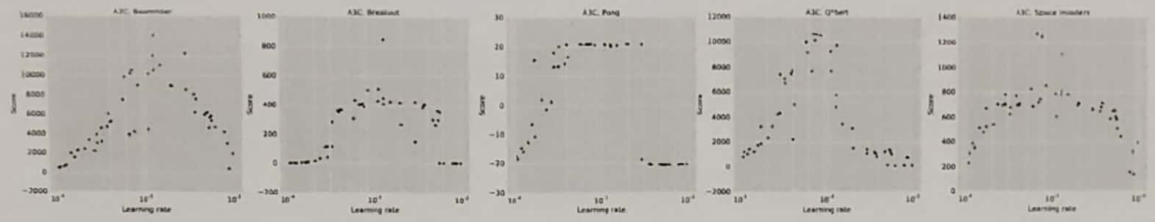


*Figure 2.* Scatter plots of scores obtained by asynchronous advantage actor-critic on five games (Beamrider, Breakout, Pong, Q*bert, Space Invaders) for 50 different learning rates and random initializations. On each game, there is a wide range of learning rates for which all random initializations acheive good scores. This shows that A3C is quite robust to learning rates and initial random weights.

numbers of actor-learners and training methods on five Atari games, and Figure 4, which shows plots of the average score against wall-clock time.

### 5.6. Robustness and Stability

Finally, we analyzed the stability and robustness of the four proposed asynchronous algorithms. For each of the four algorithms we trained models on five games (Breakout, Beamrider, Pong, Q*bert, Space Invaders) using 50 different learning rates and random initializations. Figure 2 shows scatter plots of the resulting scores for A3C, while Supplementary Figure S11 shows plots for the other three methods. There is usually a range of learning rates for each method and game combination that leads to good scores, indicating that all methods are quite robust to the choice of learning rate and random initialization. The fact that there are virtually no points with scores of 0 in regions with good learning rates indicates that the methods are stable and do not collapse or diverge once they are learning.

## 6. Conclusions and Discussion

We have presented asynchronous versions of four standard reinforcement learning algorithms and showed that they are able to train neural network controllers on a variety of domains in a stable manner. Our results show that in our proposed framework stable training of neural networks through reinforcement learning is possible with both value-based and policy-based methods, off-policy as well as on-policy methods, and in discrete as well as continuous domains. When trained on the Atari domain using 16 CPU cores, the proposed asynchronous algorithms train faster than DQN trained on an Nvidia K40 GPU, with A3C surpassing the current state-of-the-art in half the training time.

One of our main findings is that using parallel actor-learners to update a shared model had a stabilizing effect on the learning process of the three value-based methods we considered. While this shows that stable online Q-learning is possible without experience replay, which was used for this purpose in DQN, it does not mean that experience replay is not useful. Incorporating experience replay into the asynchronous reinforcement learning framework could substantially improve the data efficiency of these methods by reusing old data. This could in turn lead to much faster training times in domains like TORCS where interacting with the environment is more expensive than updating the model for the architecture we used.

Combining other existing reinforcement learning methods or recent advances in deep reinforcement learning with our asynchronous framework presents many possibilities for immediate improvements to the methods we presented. While our n-step methods operate in the *forward view* (Sutton & Barto, 1998) by using corrected n-step returns directly as targets, it has been more common to use the *backward view* to implicitly combine different returns through eligibility traces (Watkins, 1989; Sutton & Barto, 1998; Peng & Williams, 1996). The asynchronous advantage actor-critic method could be potentially improved by using other ways of estimating the advantage function, such as generalized advantage estimation of (Schulman et al., 2015b). All of the value-based methods we investigated could benefit from different ways of reducing overestimation bias of Q-values (Van Hasselt et al., 2015; Bellemare et al., 2016). Yet another, more speculative, direction is to try and combine the recent work on true online temporal difference methods (van Seijen et al., 2015) with non-linear function approximation.

In addition to these algorithmic improvements, a number of complementary improvements to the neural network architecture are possible. The dueling architecture of (Wang et al., 2015) has been shown to produce more accurate estimates of Q-values by including separate streams for the state value and advantage in the network. The spatial softmax proposed by (Levine et al., 2015) could improve both value-based and policy-based methods by making it easier for the network to represent feature coordinates.

ACKNOWLEDGMENTS

We thank Thomas Degris, Remi Munos, Marc Lanctot, Sasha Vezhnevets and Joseph Modayil for many helpful discussions, suggestions and comments on the paper. We also thank the DeepMind evaluation team for setting up the environments used to evaluate the agents in the paper.

# Supplementary Material for "Asynchronous Methods for Deep Reinforcement Learning"

June 17, 2016

## 7. Optimization Details

We investigated two different optimization algorithms with our asynchronous framework – stochastic gradient descent and RMSProp. Our implementations of these algorithms do not use any locking in order to maximize throughput when using a large number of threads.

**Momentum SGD:** The implementation of SGD in an asynchronous setting is relatively straightforward and well studied (Recht et al., 2011). Let $\theta$ be the parameter vector that is shared across all threads and let $\Delta\theta_i$ be the accumulated gradients of the loss with respect to parameters $\theta$ computed by thread number $i$. Each thread $i$ independently applies the standard momentum SGD update $m_i = \alpha m_i + (1-\alpha)\Delta\theta_i$ followed by $\theta \leftarrow \theta - \eta m_i$ with learning rate $\eta$, momentum $\alpha$ and without any locks. Note that in this setting, each thread maintains its own separate gradient and momentum vector.

**RMSProp:** While RMSProp (Tieleman & Hinton, 2012) has been widely used in the deep learning literature, it has not been extensively studied in the asynchronous optimization setting. The standard non-centered RMSProp update is given by

$$g = \alpha g + (1-\alpha)\Delta\theta^2 \tag{S2}$$

$$\theta \leftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g+\epsilon}}, \tag{S3}$$

where all operations are performed elementwise. In order to apply RMSProp in the asynchronous optimization setting one must decide whether the moving average of elementwise squared gradients $g$ is shared or per-thread. We experimented with two versions of the algorithm. In one version, which we refer to as RMSProp, each thread maintains its own $g$ shown in Equation S2. In the other version, which we call Shared RMSProp, the vector $g$ is shared among threads and is updated asynchronously and without locking. Sharing statistics among threads also reduces memory requirements by using one fewer copy of the parameter vector per thread.

We compared these three asynchronous optimization algorithms in terms of their sensitivity to different learning rates and random network initializations. Figure S5 shows a comparison of the methods for two different reinforcement learning methods (Async $n$-step Q and Async Advantage Actor-Critic) on four different games (Breakout, Beamrider, Seaquest and Space Invaders). Each curve shows the scores for 50 experiments that correspond to 50 different random learning rates and initializations. The x-axis shows the rank of the model after sorting in descending order by final average score and the y-axis shows the final average score achieved by the corresponding model. In this representation, the algorithm that performs better would achieve higher maximum rewards on the y-axis and the algorithm that is most robust would have its slope closest to horizontal, thus maximizing the area under the curve. RMSProp with shared statistics tends to be more robust than RMSProp with per-thread statistics, which is in turn more robust than Momentum SGD.

case the we used a cost on the differential entropy of the normal distribution defined by the output of the actor network, $-\frac{1}{2}(\log(2\pi\sigma^2)+1)$, we used a constant multiplier of $10^{-4}$ for this cost across all of the tasks examined. The asynchronous advantage actor-critic algorithm finds solutions for all the domains. Figure S8 shows learning curves against wall-clock time, and demonstrates that most of the domains from states can be solved within a few hours. All of the experiments, including those done from pixel based observations, were run on CPU. Even in the case of solving the domains directly from pixel inputs we found that it was possible to reliably discover solutions within 24 hours. Figure S7 shows scatter plots of the top scores against the sampled learning rates. In most of the domains there is large range of learning rates that consistently achieve good performance on the task.

**Algorithm S2** Asynchronous n-step Q-learning - pseudocode for each actor-learner thread.

*// Assume global shared parameter vector $\theta$.*
*// Assume global shared target parameter vector $\theta^-$.*
*// Assume global shared counter $T = 0$.*
Initialize thread step counter $t \leftarrow 1$
Initialize target network parameters $\theta^- \leftarrow \theta$
Initialize thread-specific parameters $\theta' = \theta$
Initialize network gradients $d\theta \leftarrow 0$
**repeat**
  Clear gradients $d\theta \leftarrow 0$
  Synchronize thread-specific parameters $\theta' = \theta$
  $t_{start} = t$
  Get state $s_t$
  **repeat**
    Take action $a_t$ according to the $\epsilon$-greedy policy based on $Q(s_t, a; \theta')$
    Receive reward $r_t$ and new state $s_{t+1}$
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  **until** terminal $s_t$ **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ \max_a Q(s_t, a; \theta^-) & \text{for non-terminal } s_t \end{cases}$
  **for** $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt $\theta'$: $d\theta \leftarrow d\theta + \frac{\partial (R - Q(s_i, a_i; \theta'))^2}{\partial \theta'}$
  **end for**
  Perform asynchronous update of $\theta$ using $d\theta$.
  **if** $T \mod I_{target} == 0$ **then**
    $\theta^- \leftarrow \theta$
  **end if**
**until** $T > T_{max}$

A3C

**Algorithm S3** Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

*// Assume global shared parameter vectors $\theta$ and $\theta_v$ and global shared counter $T = 0$*
*// Assume thread-specific parameter vectors $\theta'$ and $\theta'_v$*
Initialize thread step counter $t \leftarrow 1$
**repeat**
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$
  $t_{start} = t$
  Get state $s_t$
  **repeat**
    Perform $a_t$ according to policy $\pi(a_t|s_t;\theta')$
    Receive reward $r_t$ and new state $s_{t+1}$
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  **until** terminal $s_t$ **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \text{// Bootstrap from last state} \end{cases}$
  **for** $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt $\theta'$: $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i;\theta')(R - V(s_i;\theta'_v))$
    Accumulate gradients wrt $\theta'_v$: $d\theta_v \leftarrow d\theta_v + \partial\,(R - V(s_i;\theta'_v))^2/\partial\theta'_v$
  **end for**
  Perform asynchronous update of $\theta$ using $d\theta$ and of $\theta_v$ using $d\theta_v$.
**until** $T > T_{max}$